

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING IN POLYMER INFORMATICS

Day 1

Structure Representation and Feature Engineering

Tianle Yue (tyue4@wisc.edu)

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING IN POLYMER INFORMATICS

Day 1 Structure Representation and Feature Engineering

Day 2 Supervised Learning and Machine Learning Models

Day 3 Unsupervised Learning and Explainable ML

Day 4 Generative Models and Reinforcement Learning

Code Link

- <https://github.com/ytl0410/Polymer-Informatics>
- https://drive.google.com/drive/folders/1blO2hW-e2XyUtUVUnKVJMNgO9eleLPDu?usp=drive_link
- https://uwprod-my.sharepoint.com/:u:/g/personal/tyue4_wisc_edu/EYDJnNn5xK9GjfSJYCcbqwUBn7Xm_VLcK_rXSIhrt-MHGO?e=6gMDsX



Python Environment:

- `conda create -n py39 python=3.9`
- `conda install -c conda-forge cudatoolkit=11.2 cudnn=8.1.0`
- `python -m pip install "tensorflow<2.11"`
- `pip install numpy==1.26.4`
- `pip install matplotlib==3.8.0`
- `pip install pandas==1.5.3`
- `pip install rdkit==2023.9.1`
- `pip install scikit-learn==1.3.0`
- `pip install keras-tuner==1.4.5`
- `pip install seaborn==0.13.0`



Open-Source Cheminformatics
and Machine Learning

<https://www.rdkit.org/docs/Install.html>

With pip (in Google Colab):

- `pip install rdkit`



TensorFlow

OR



<https://www.tensorflow.org/install>

Version	Python version	Compiler	Build tools	cuDNN	CUDA
tensorflow-2.7.0	3.7-3.9	GCC 7.3.1	Bazel 3.7.2	8.1	11.2
tensorflow-2.6.0	3.6-3.9	GCC 7.3.1	Bazel 3.7.2	8.1	11.2
tensorflow-2.5.0	3.6-3.9	GCC 7.3.1	Bazel 3.7.2	8.1	11.2
tensorflow-2.4.0	3.6-3.8	GCC 7.3.1	Bazel 3.1.0	8.0	11.0
tensorflow-2.3.0	3.5-3.8	GCC 7.3.1	Bazel 3.1.0	7.6	10.1
tensorflow-2.2.0	3.5-3.8	GCC 7.3.1	Bazel 2.0.0	7.6	10.1

Go to Google Drive and log in

<https://workspace.google.com/products/drive/>



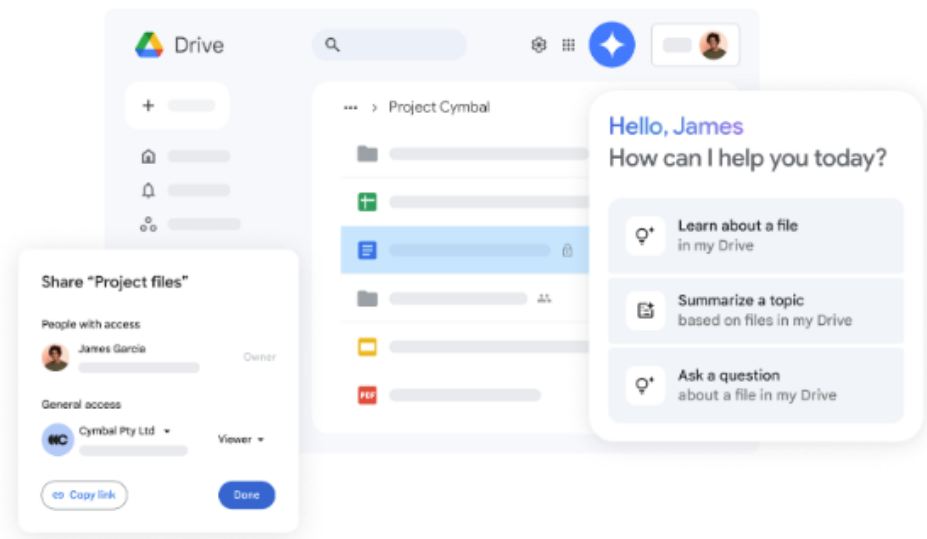
Google Drive

Store and share files online

AI-powered cloud storage for seamless file sharing and enhanced collaboration.

Sign in

Try Drive for work



Create a new folder

1

My Drive

2

My Drive

3

New folder

File upload

Folder upload

Google Docs

Google Sheets

Google Slides

Google Vids

Google Forms

More

My Drive

Type | People | Modified | Source

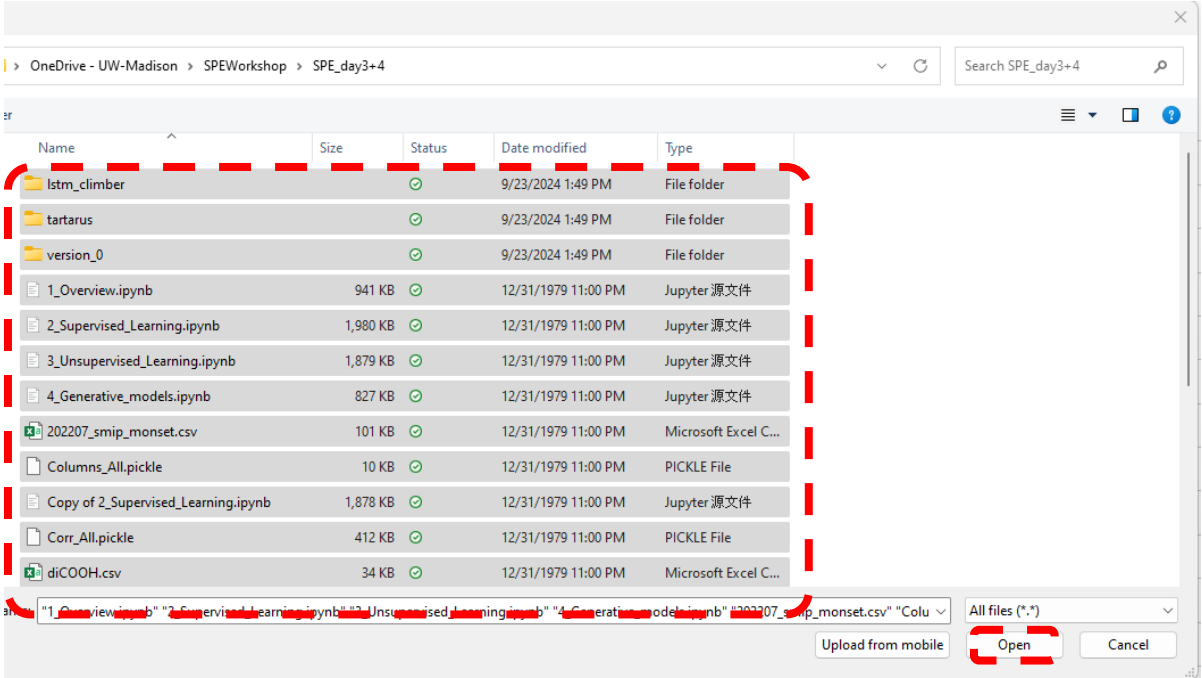
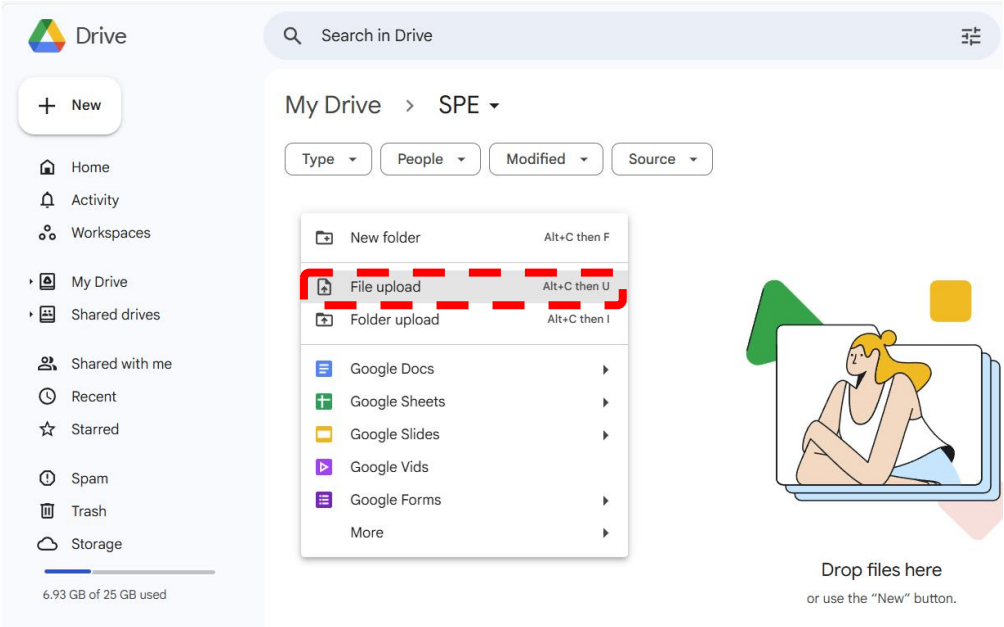
Name	Owner
Colab	me
Module 4_informatics	
Modules	
new	
SPE	
SPE_1	

New folder

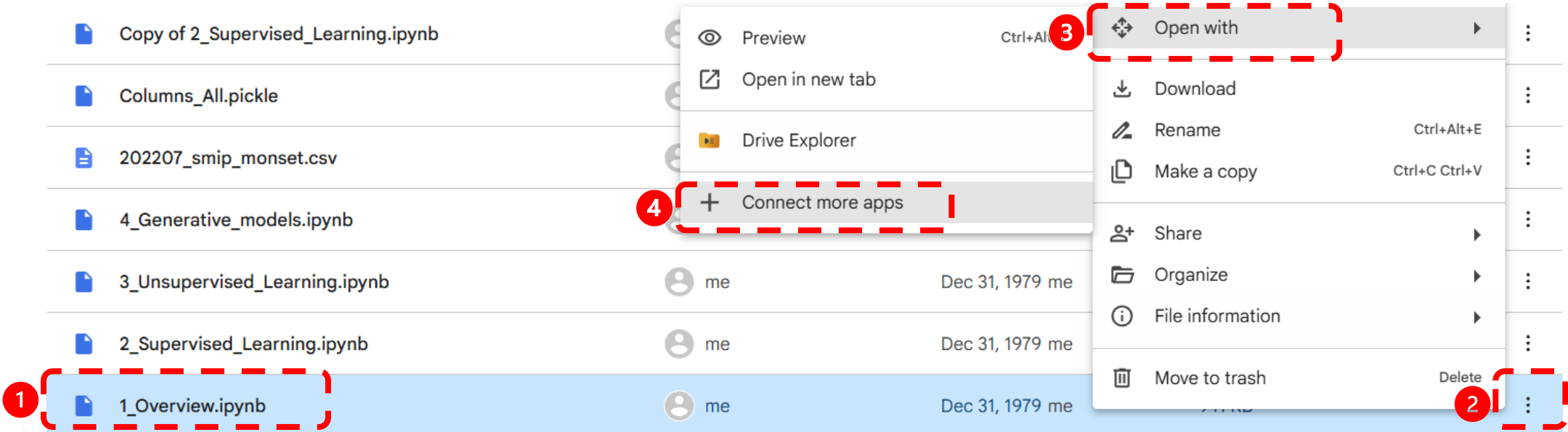
SPE

Cancel | Create

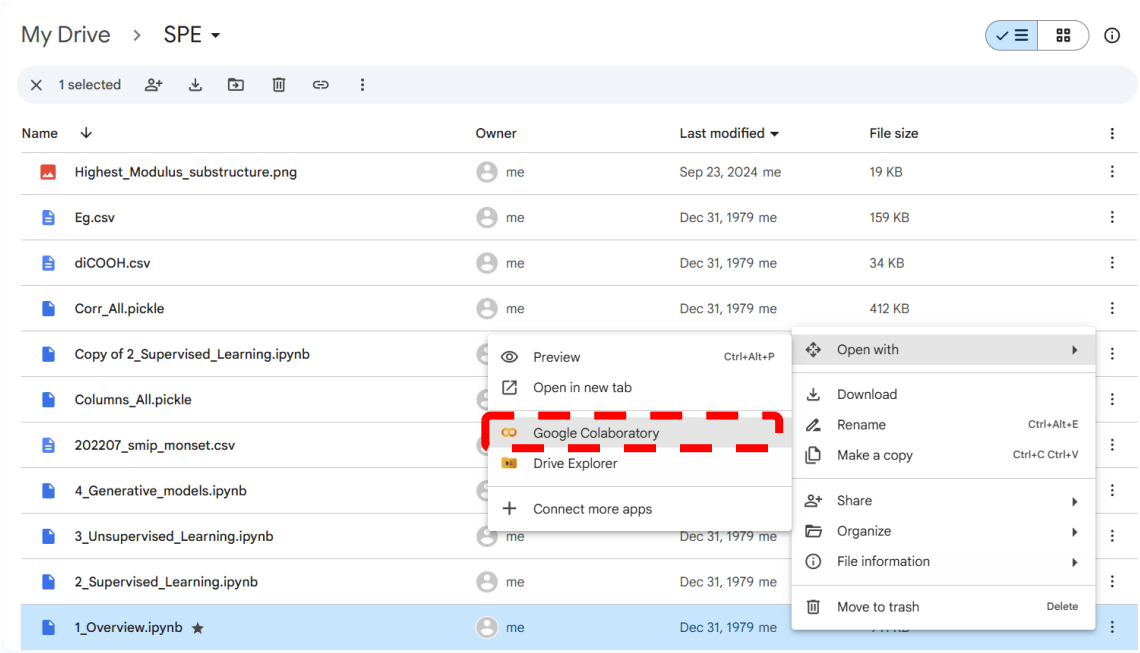
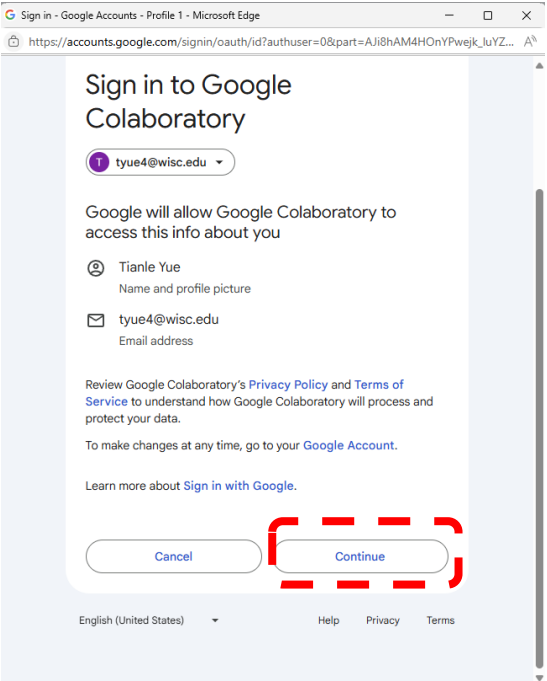
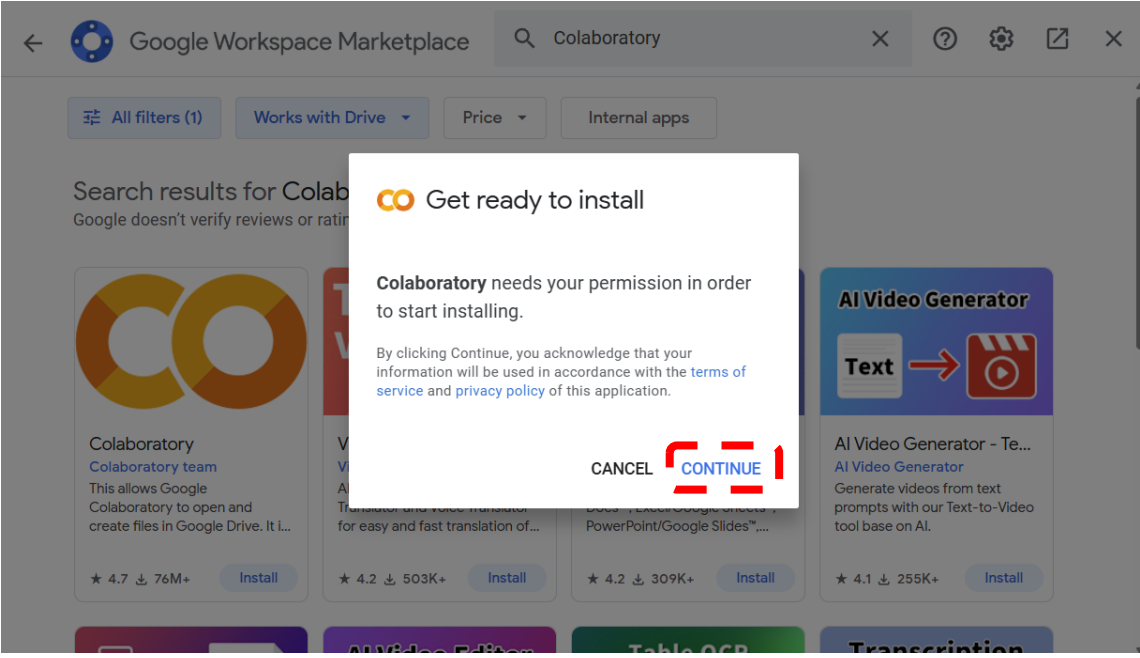
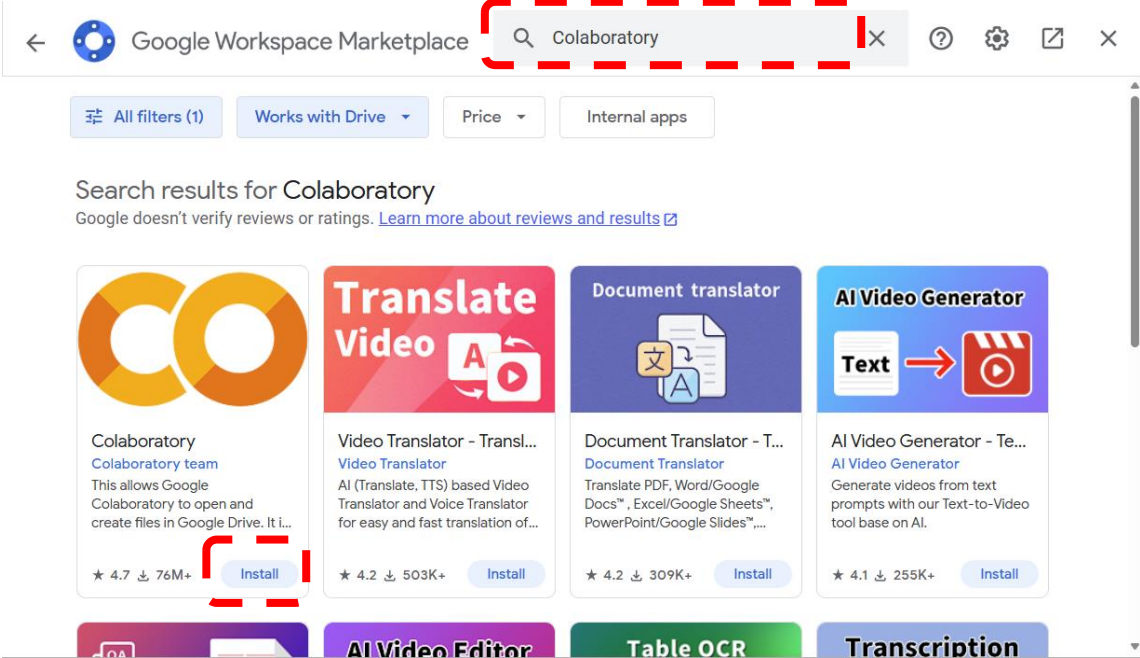
Upload the code and dataset files into the new folder



Connect to Google Colab



Connect to Google Colab



Open the code file

 1_Overview.ipynb  

File Edit View Insert Runtime Tools Help

Share Gemini T

Q Commands + Code + Text ▶ Run all ▼ Connect ▼

Google Colab Specific Setup

For getting files on Google Drive and to Open in Colab

First download the files from the lab and upload them to google drive near the top level. Then go to this folder in your browser and click on column of dots to the right of the *.ipynb file. Select "Open With". If "Google Collaboratory" appears choose it. If it does not appear go to the bottom of the list and choose "Connect more apps" and then install "Google Collaboratory". Close browser, reopen, and try again. Now "Google Collaboratory" should appear when you select "Open With". You should also just be able to double click on the *.ipynb file.

↑ ↓ ↻ ⌨ ✎ 📄 🗑 ⋮

Markdown cell

This cell links your Google drive to the Colab session



```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Run the code in the cell

Here, modify the path where you saved this notebook and the associated lab files on your Google drive. This can be obtained in colab

```
[ ] path = '/content/drive/MyDrive/SPE/'
```

Check the path name is assigned correctly.

Links your Google drive to the Colab session

```
[ ] from google.colab import drive
drive.mount('/content/drive')
```

Permit this notebook to access your Google Drive files?

This notebook is requesting access to your Google Drive files. Granting access to Google Drive will permit code executed in the notebook to modify files in your Google Drive. Make sure to review notebook code prior to allowing this access.

No thanks **Connect to Google Drive**

Sign in with Google

You're signing back in to Google Drive for desktop

Review Google Drive for desktop's [Privacy Policy](#) and [Terms of Service](#) to understand how Google Drive for desktop will process and protect your data.

To make changes at any time, go to your [Google Account](#).

Learn more about [Sign in with Google](#).

tyue4@wisc.edu

Cancel **Continue**

Make sure you trust Google Drive for desktop

Review Google Drive for desktop's [Privacy Policy](#) and [Terms of Service](#) to understand how Google Drive for desktop will process and protect your data.

To make changes at any time, go to your [Google Account](#).

Learn how Google helps you [share data safely](#).

Cancel **Continue**

Runtime duration

1m

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Output

Links your Google drive to the Colab session

✓
0s

[2] path = `'/content/drive/MyDrive/SPE/'`  **Modify the path to the directory where your files are stored**

Check the path name is assigned correctly.

✓
0s

[3] `!echo $path`
`import os`
`os.path.isdir(path)`

 `/content/drive/MyDrive/SPE/`
`True`

 **The path is set correctly**

✓
0s

 `# This is needed so can import packages from a different path than standard libraries`
`import sys`
`sys.path.append(path)`

Check the Python packages that are available by default in the Colab environment.

Set up Python environment

Pre-installed packages 

 `!pip list`

Package	Version

absl-py	1.4.0
accelerate	0.32.1
aiohappyeyeballs	2.4.0
aiohttp	3.10.5
aiosignal	1.3.1
alabaster	0.7.16
albucore	0.0.13
albumations	1.4.14
altair	4.2.2
annotated-types	0.7.0
anyio	3.7.1
argon2-cffi	23.1.0
argon2-cffi-bindings	21.2.0
array_record	0.5.1
arviz	0.18.0
asn1crypto	1.5.1
astropy	6.1.2
astropy-iers-data	0.2024.8.26.0.31.57

Need to install some python packages which is not present on Colab environment by default.

✓
11s

 `!pip install rdkit`

 Collecting rdkit
 Downloading rdkit-2025.3.5-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (4.1 kB)
 Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from rdkit) (2.0.2)
 Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from rdkit) (11.3.0)
 Downloading rdkit-2025.3.5-cp312-cp312-manylinux_2_28_x86_64.whl (36.2 MB)
 ----- 36.2/36.2 MB 51.3 MB/s eta 0:00:00

 Installing collected packages: rdkit
 Successfully installed rdkit-2025.3.5

 **Install rdkit package successfully**

Agentic AI/ skills



Codex

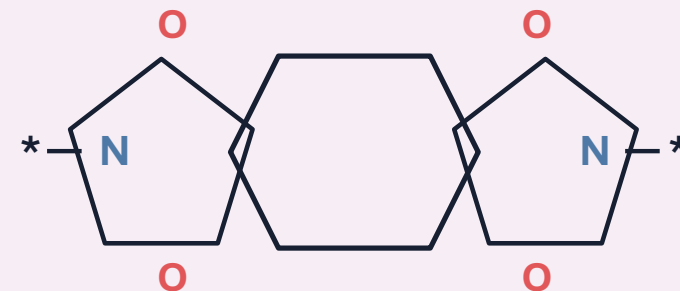
The same powerful coding agent—now in ChatGPT.

Name	Last commit message	Last commit date
..		
agents	Keep only workshop skills and appen...	1 minute ago
references	Keep only workshop skills and appen...	1 minute ago
scripts	Keep only workshop skills and appen...	1 minute ago
tests	Keep only workshop skills and appen...	1 minute ago
LICENSE	Keep only workshop skills and appen...	1 minute ago
SKILL.md	Keep only workshop skills and appen...	1 minute ago



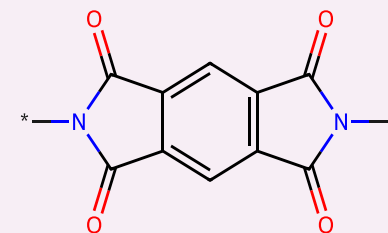
User

Draw molecule structure of
N1C(=O)C2=CC3=C(C=C2C1=O)C(=O)N()C3=O



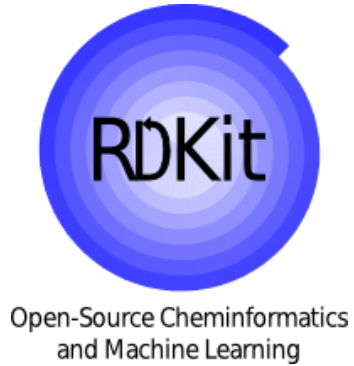
User

Draw molecule structure of
N1C(=O)C2=CC3=C(C=C2C1=O)C(=O)N()C3=O
with Rdkit



Python packages

RDKit: polymer (chemical) informatics



- Morgan fingerprint generation
- Molecular descriptor calculation
- Molecular structure visualization
- SMILES parsing and validation
- Substructure search
- Molecular similarity analysis
- Rule-based polymer structure generation

Scikit-learn: classical machine learning algorithms



- Dataset splitting
- Model evaluation and cross-validation
- Clustering analysis
- Feature scaling
- Dimensionality reduction

TensorFlow: deep learning framework



Matplotlib: Visualization with Python



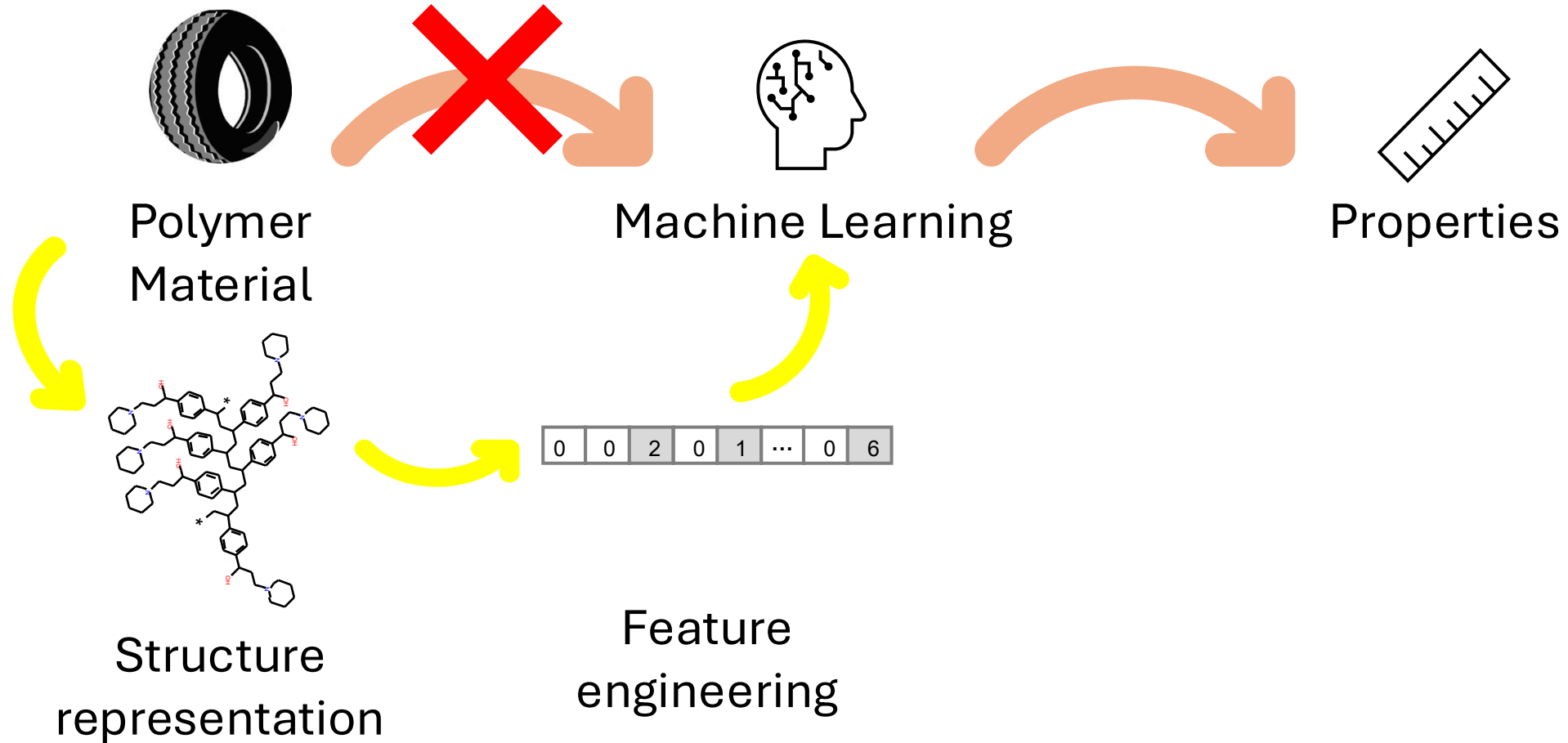
PyTorch: deep learning and generative AI



Pandas: data analysis and manipulation



Structure Representation and Feature Engineering



Import all required packages

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

import rdkit
from rdkit import Chem
from rdkit.Chem import AllChem
from rdkit.Chem import Descriptors
from rdkit.Chem import rdMolDescriptors
from rdkit.Chem.Draw import IPythonConsole
from rdkit.Chem import Draw

from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.svm import SVR
from sklearn.linear_model import Lasso, LassoCV
from sklearn.model_selection import train_test_split, cross_val_predict, cross_validate
from sklearn.preprocessing import StandardScaler
from sklearn.utils import shuffle
from math import sqrt

from tensorflow.keras.models import Sequential, save_model, load_model
from tensorflow.keras.layers import Dense, Activation, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow import keras
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Conv1D, MaxPooling1D, Dense, Flatten, Activation
import time
from sklearn.metrics import r2_score

from collections import Counter
import pickle
import pandas as pd
import collections
import seaborn as sns

from rdkit import rdBase
rdBase.DisableLog('rdApp.warning')
```

Scientific computing package

Plotting

Data management

Rdkit: Cheminformatics toolkit

scikit-learn
Library for classical machine learning algorithms

Deep learning framework

Importing the Dataset

```
[8] # Read in the Tg data  
Tg_df = pd.read_csv(os.path.join(path, "Tg.csv"))
```

→ Dataset file name

```
# View the dataset (dataframe)  
Tg_df
```



Smiles Tg(C)



→ Interactive table



→ Chart

0	*C*	-63.48
1	*C(C*)C	-2.73
2	*C(C*)CC	-22.54
3	*C(C*)CCC	-32.29
4	*C(C*)C(C)C	10.97
...	...	...
6901	*C1C(C(C(C1)(O)C(F)(F)F)(F)F)(C(C*)(F)F)F	152.00
6902	*C1C(CC(C1)CC(O)(C(F)(F)F)C(F)(F)F)(C(C*)(F)F)F	98.00
6903	*C1C(CC(C1)C(O)(C(F)(F)F)C(F)(F)F)(C(C*)(F)F)F	118.00
6904	*c1cc2C3(CCC(CC3)(C3=NC(CO3)c3ccccc3)C3=NC(CO3...	161.00
6905	*c1cc2C3(CCC(CC3)(C3=NC(CO3)Cc3ccccc3)C3=NC(CO...	142.00

6906 rows x 2 columns

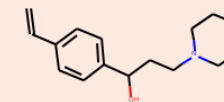
↓
Dataset size

↓
First column:
Smiles of repeat units

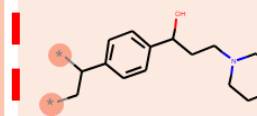
↓
Second column: label
values / attribute values

Structure

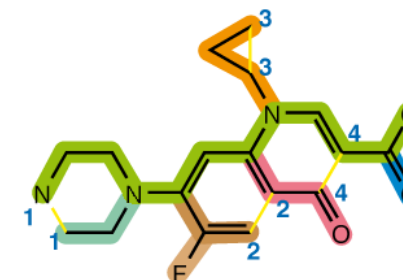
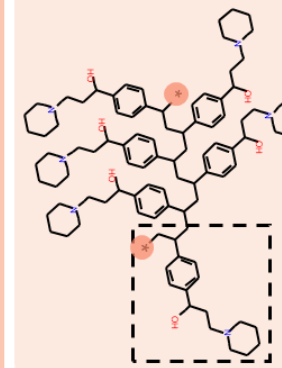
Monomer



Repeat unit



Repeat units polymerized



N1CCN(CC1)C(C(F)=C2)=CC(=C2C4=O)N(C3CC3)C=C4C(=O)O

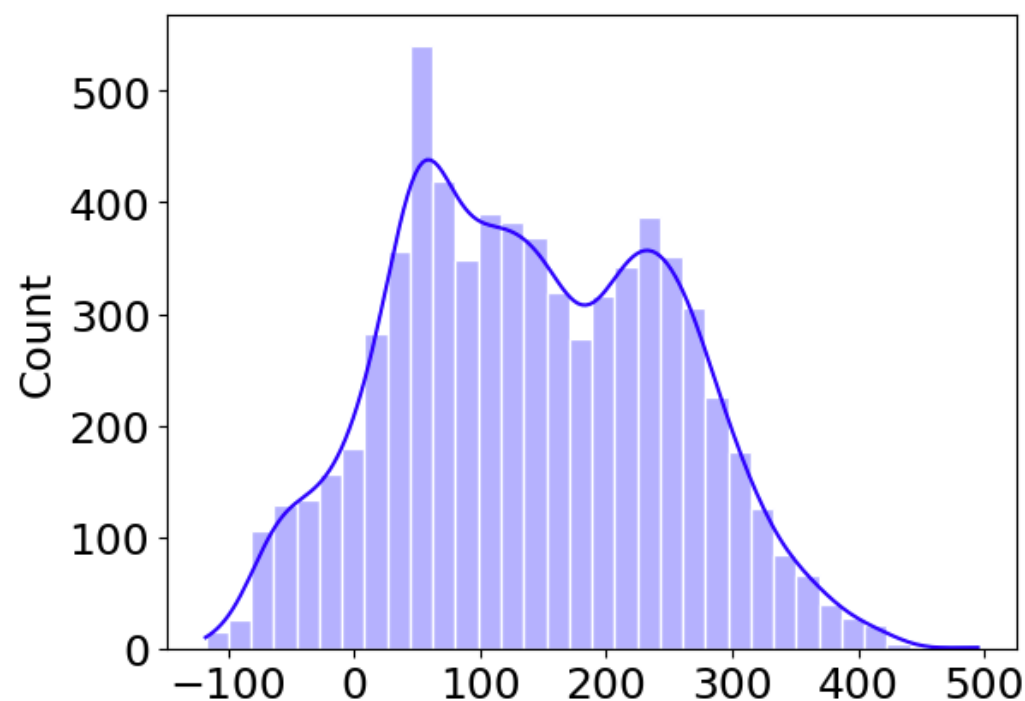
View dataset information

```
[ ] # View the distribution of dataset labels
# Set keyword arguments for the histogram plot, including transparency and edge color
kwargs = dict(hist_kws={'alpha':.3, 'edgecolor':'white'})

# Plot the distribution of the 'Tg(C)' column from Tg_df
sns.histplot(Tg_df['Tg(C)'].dropna(), kde=True, color = 'blue', alpha = 0.3, edgecolor='white')
# Set the font size for the x and y axis ticks
plt.xticks(size=18)
plt.yticks(size=18)
plt.xlabel("'C'", fontsize=18)
plt.ylabel("Count", fontsize=18)
```

Column name

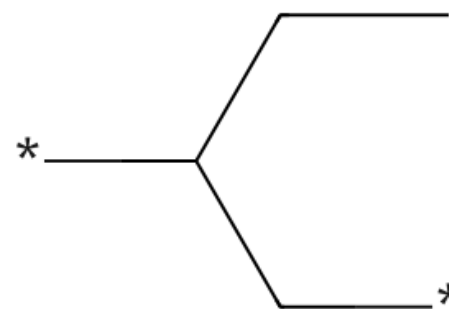
Text(0, 0.5, 'Count')



```
[ ] # View molecular structures in the dataset
smiles = Tg_df['Smiles'][2]
print(smiles)
molecule = Chem.MolFromSmiles(smiles)
Draw.MolToImage(molecule)
```

Sample index
(start from 0)

C(C)CC



Morgan fingerprint

Search radius (Generally set to 3)

```
# Generation of Morgan fingerprint

fn = Tg_df.Smiles.apply(Chem.MolFromSmiles).apply(lambda m: AllChem.GetMorganFingerprintAsBitVect(m, radius=3, nBits=1024))

# Convert the Morgan fingerprints into a DataFrame with 1024 columns.
MF = pd.DataFrame()
columns = []
for i in range(1024):
    col_name = f'Bit_{i+1}'
    new_column = fn.apply(lambda x: int(bool(x.GetBit(i))))
    columns.append(pd.Series(new_column, name=col_name))
MF = pd.concat([MF] + columns, axis=1)
MF
```

The length of the fingerprint bit vector (Generally set to 1024 or 2048)

	Bit_1	Bit_2	Bit_3	Bit_4	Bit_5	Bit_6	Bit_7	Bit_8	Bit_9	Bit_10	...	Bit_1015	Bit_1016	Bit_1017	Bit_1018	Bit_1019	Bit_1020	Bit_1021	Bit_1022	Bit_1023	Bit_1024
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	1	1	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	0	1	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	1	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6901	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	0
6902	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	0
6903	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	0
6904	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	0
6905	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	0

6906 rows x 1024 columns

The generated Morgan fingerprint

View the substructure represented by each bit of the Morgan Fingerprint

0s

```
# View the substructure represented by each bit of the Morgan Fingerprint

# Select a specific bit
bit = 0

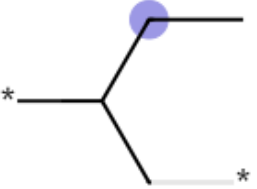
# Loop through the first 6000 molecules in the dataset
for i in range(6000):
    # Convert the SMILES string to a molecule object
    mol = Chem.MolFromSmiles(Tg_df.loc[i, 'Smiles'])
    # Get the bit information of the Morgan fingerprint
    info={}
    fp = rdMolDescriptors.GetMorganFingerprintAsBitVect(mol, 3, nBits=1024, bitInfo=info)
    # Check if the selected bit is present in the fingerprint information
    if bit in info:

        submols = []
        for atom_idx, radius in info[bit]:
            if 1:
                env = Chem.FindAtomEnvironmentOfRadiusN(mol, radius, atom_idx)
                atoms = set(Chem.FindAtomEnvironmentOfRadiusN(mol, radius, atom_idx))
                submols.append(Chem.PathToSubmol(mol, env))

        # Display the substructures corresponding to the selected bit
        for submol in submols:
            display(Draw.DrawMorganBit(mol, bit, info))

        break
```

⇒



Bit index (start from 0)

The loop range in the dataset (determined by the dataset's size)

bit = 0



⇒

	Bit_1	Bit_2	Bit_3	Bit_4	Bit_5	Bit_6	Bit_7	Bit_8	Bit_9	Bit_10
0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0
2	1	1	0	0	0	0	0	0	0	0
3	0	1	0	0	0	0	0	0	0	0
4	0	1	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...
6901	0	0	0	0	0	0	0	0	0	0
6902	0	0	0	0	0	0	0	0	0	0
6903	0	0	0	0	0	0	0	0	0	0
6904	0	0	0	0	0	0	0	0	0	0
6905	0	0	0	0	0	0	0	0	0	0

6906 rows x 1024 columns

View the substructure represented by each bit of the Morgan Fingerprint

```
# View the substructure represented by each bit of the Morgan Fingerprint

# Select a specific bit
bit = 0

# Loop through the first 6000 molecules in the dataset
for i in range(6000):
    # Convert the SMILES string to a molecule object
    mol = Chem.MolFromSmiles(Tg_df.loc[i, 'Smiles'])
    # Get the bit information of the Morgan fingerprint
    info={}
    fp = rdMolDescriptors.GetMorganFingerprintAsBitVect(mol, 3, nBits=1024, bitInfo=info)
    # Check if the selected bit is present in the fingerprint information
    if bit in info:

        submols = []
        for atom_idx, radius in info[bit].items():
            if 1:
                env = Chem.FindAtomSubstructure(mol, atom_idx, radius)
                atoms = set([Che
                submols.append(

# Display the substructure
for submol in submols:
    display(DrawMc

break
```

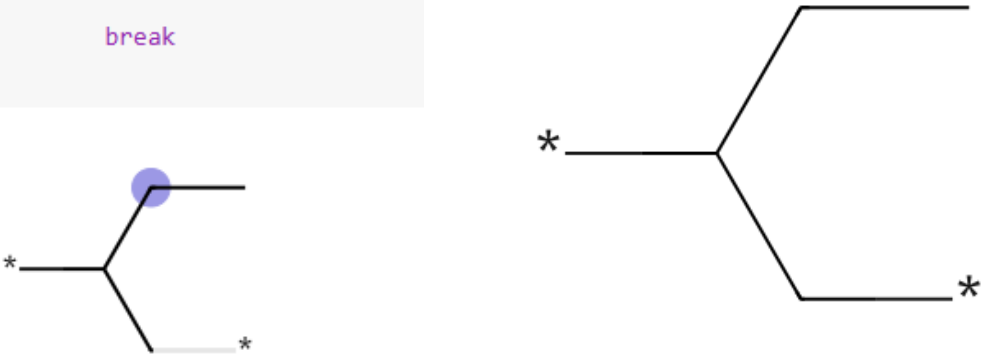
Bit index (start from 0)

The loop range in the dataset (determined by the dataset's size)

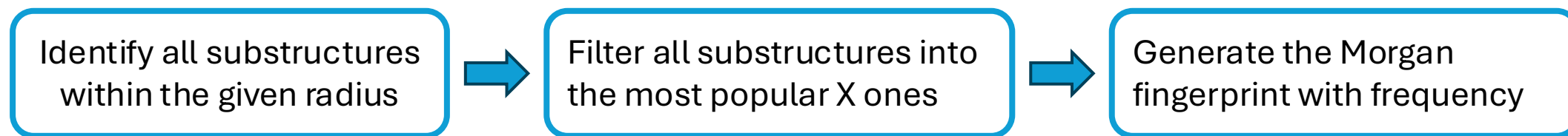
bit = 0

	Bit_1	Bit_2	Bit_3	Bit_4	Bit_5	Bit_6	Bit_7	Bit_8	Bit_9	Bit_10
0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0
2	1	1	0	0	0	0	0	0	0	0
3	0	1	0	0	0	0	0	0	0	0
4	0	1	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...
6901	0	0	0	0	0	0	0	0	0	0
6902	0	0	0	0	0	0	0	0	0	0
6903	0	0	0	0	0	0	0	0	0	0
6904	0	0	0	0	0	0	0	0	0	0
6905	0	0	0	0	0	0	0	0	0	0

6906 rows x 1024 columns



Morgan fingerprint with frequency



Morgan fingerprint with frequency

Identify all substructures within the given radius

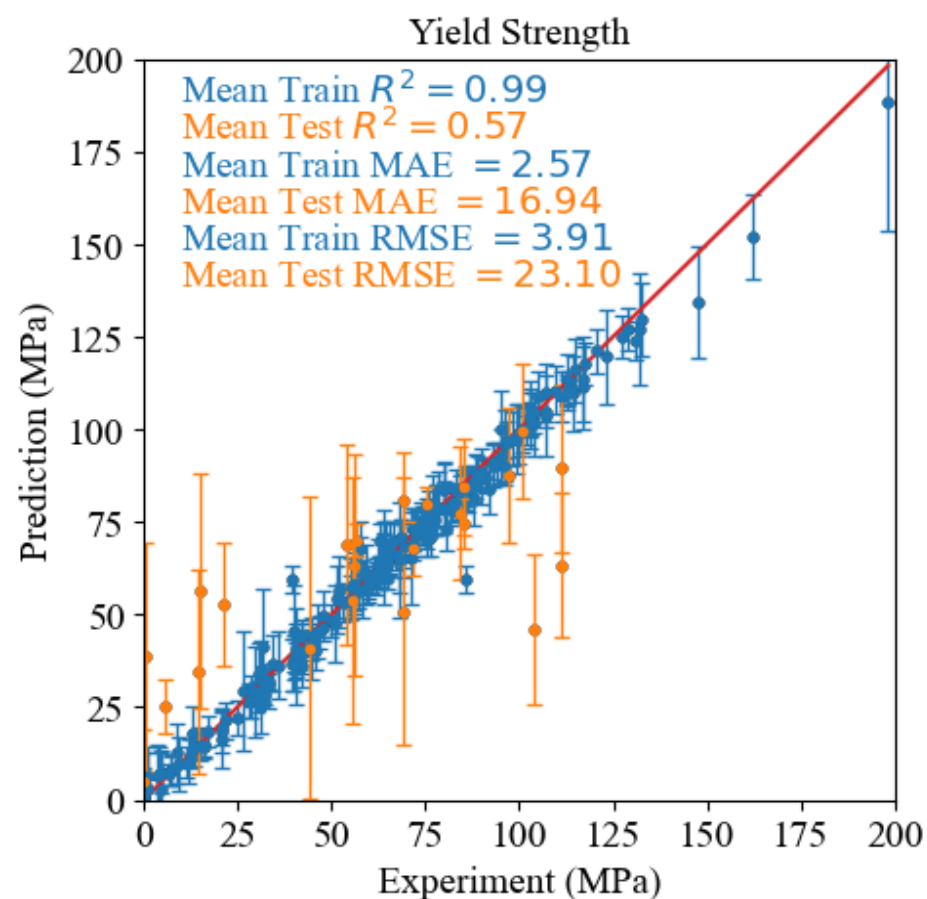


Filter all substructures into the most popular X ones

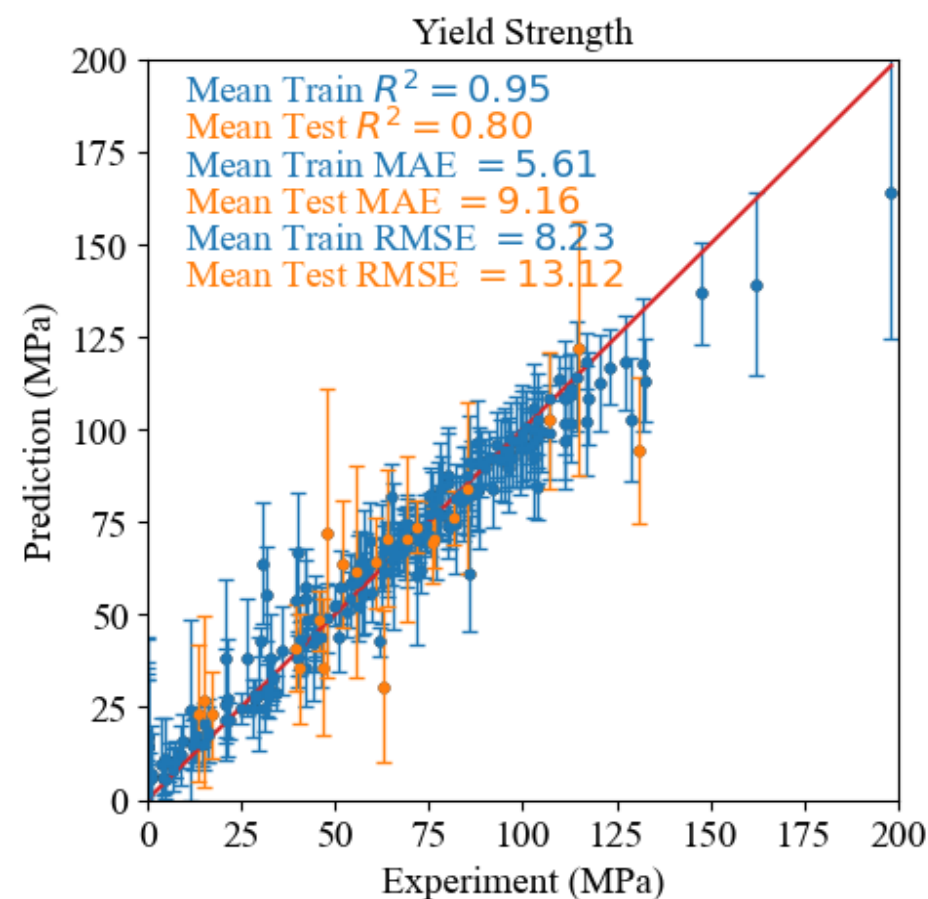


Generate the Morgan fingerprint with frequency

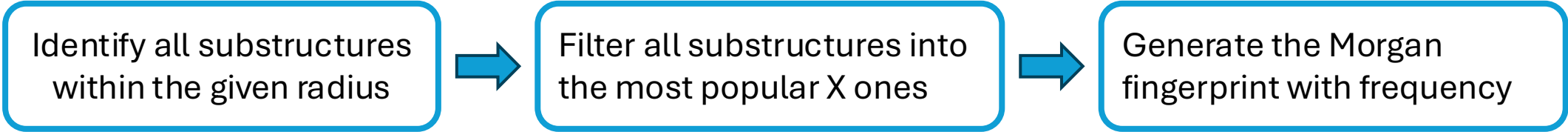
129-bit Morgan fingerprints



129-bit Morgan fingerprints with frequency



Morgan fingerprint with frequency



```
[ ] # Count all the substructure information

# Convert SMILES strings in 'Tg_df.Smiles' to RDKit molecule objects
molecules = Tg_df.Smiles.apply(Chem.MolFromSmiles)

# Generate Morgan fingerprints for each molecule with a radius of 3
fp = molecules.apply(lambda m: AllChem.GetMorganFingerprint(m, radius=3))

# Get the non-zero elements from the fingerprints (i.e., the bits that are set)
fp_n = fp.apply(lambda m: m.GetNonzeroElements())

# Initialize an empty list to store the hash codes (bit identifiers)
HashCode = []
for i in fp_n:
    for j in i.keys():
        HashCode.append(j)

# Get unique hash codes and convert them into a list
unique_set = set(HashCode)
unique_list = list(unique_set)

# Create a DataFrame to map unique hash codes to their indices
Corr_df = pd.DataFrame(unique_list).reset_index()

# Initialize an empty list to store the fingerprint vectors
MY_finger = []
for polymer in fp_n:
    my_finger = [0] * len(unique_list)
    for key in polymer.keys():
        index = Corr_df[Corr_df[0] == key]['index'].values[0]
        my_finger[index] = polymer[key]
    MY_finger.append(my_finger)

# Convert the list of fingerprint vectors into a DataFrame
MFF = pd.DataFrame(MY_finger)
MFF
```

Search radius (Generally set to 3)

	0	1	2	3	4	5	6	7	8	9	...	26281	26282	26283	26284	26285	26286	26287	26288	26289	26290
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6901	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6902	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6903	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6904	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6905	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

6906 rows x 26291 columns



Total number of substructures identified (too large)

Morgan fingerprint with frequency

Identify all substructures within the given radius



Filter all substructures into the most popular X ones



Generate the Morgan fingerprint with frequency

```
✓ [18] # filter input into the most popular X substructures
0s
# Count the number of zeros in each column of the DataFrame
Zero_Sum = (MFF == 0).astype(int).sum()

# Set a threshold for the number of zeros in a column
NumberOfZero = 6870

# Print the number of columns with fewer zeros than the threshold
print(len(Zero_Sum[Zero_Sum < NumberOfZero]))
```

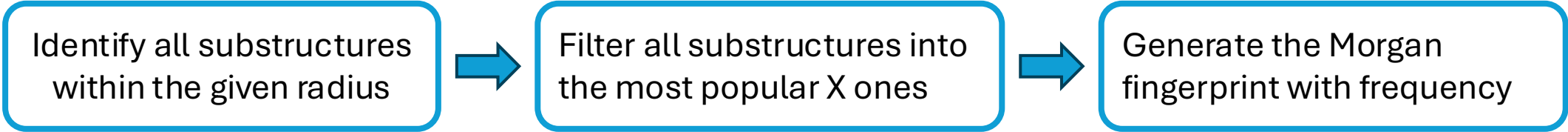
Threshold for the maximum number of samples that do not contain a given substructure

1210



The number of substructures obtained after filtering

Morgan fingerprint with frequency



```
# Generate the Morgan fingerprint with frequency after determining the number of dimensions

Columns = Zero_Sum[Zero_Sum < NumberOfZero].index
Substructure_list = list(polymer.keys())
MFF = MFF[Columns]
MFF
```

	15	36	127	142	169	205	210	240	243	259	...	26115	26117	26127	26137	26154	26162	26199	26215	26231	26286
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6901	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6902	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6903	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6904	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6905	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

Indexes of the substructures



The generated Morgan fingerprint with frequency

View the substructure represented by each bit of the Morgan fingerprint with frequency

```
# View the substructure represented by each bit of the Morgan Fingerprint

# Select a specific bit
bit = 399

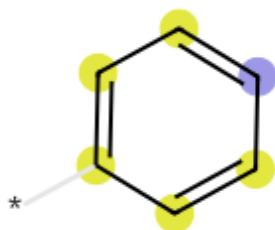
for i in range(6000):
    mol = Chem.MolFromSmiles(Tg_df.loc[i,'Smiles'])
    # Get the bit information of the Morgan fingerprint
    bi = {}
    fp = rdMolDescriptors.GetMorganFingerprint(mol, radius=3, bitInfo=bi)
    fp_n = fp.GetNonzeroElements()
    Code = (Corr_df[Corr_df['index'] == bit][0].values[0])
    if Code in fp_n.keys():
        img = Draw.DrawMorganBit(mol, Code, bi)

        break

img
```

Bit index (start from 0)

The loop range in the dataset (determined by the dataset's size)



View the substructure represented by each bit of the Morgan fingerprint with frequency

```
# View the substructure represented by each bit of the Morgan Fingerprint

# Select a specific bit
bit = 399

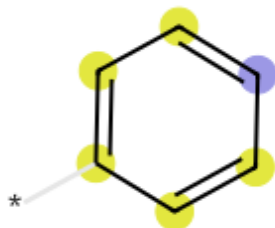
for i in range(6000):
    mol = Chem.MolFromSmiles(Tg_df.loc[i,'Smiles'])
    # Get the bit information of the Morgan fingerprint
    bi = {}
    fp = rdMolDescriptors.GetMorganFingerprint(mol, radius=3, bitInfo=bi)
    fp_n = fp.GetNonzeroElements()
    Code = (Corr_df[Corr_df['index'] == bit][0].values[0])
    if Code in fp_n.keys():
        img = Draw.DrawMorganBit(mol, Code, bi)

        break

img
```

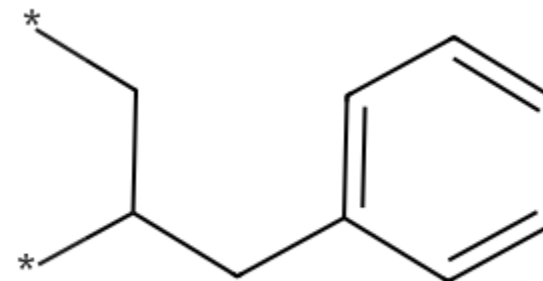
Bit index (start from 0)

The loop range in the dataset (determined by the dataset's size)



```
# View molecular structures in the dataset
smiles = Tg_df['Smiles'][21]
print(smiles)
molecule = Chem.MolFromSmiles(smiles)
Draw.MolToImage(molecule)
```

C(C)Cc1ccccc1



Morgan fingerprint with frequency

Identify all substructures within the given radius



Filter all substructures into the most popular X ones



Generate the Morgan fingerprint with frequency

```
[ ] #Save the generated information for future reproduction of the same Morgan fingerprint with frequency.
```

```
pickle_out = open("Corr_All.pickle","wb")  
pickle.dump(Corr_df, pickle_out)  
pickle_out.close()
```

```
pickle_out = open("unique_list_All.pickle","wb")  
pickle.dump(unique_list, pickle_out)  
pickle_out.close()
```

```
pickle_out = open("polymer.keys_All.pickle","wb")  
pickle.dump(Substructure_list, pickle_out)  
pickle_out.close()
```

```
pickle_out = open("Columns_All.pickle","wb")  
pickle.dump(Columns, pickle_out)  
pickle_out.close()
```



Save the Morgan fingerprints with frequency along with their frequency, ensure we can regenerate them later

Descriptors

```
✓ [24] def calculate_all_descriptors(smiles):  
3m      """  
      Calculate all molecular descriptors for a molecule given its SMILES representation.  
  
      Parameters:  
      smiles (str): The SMILES string representing the molecule.  
  
      Returns:  
      dict: A dictionary containing all the calculated molecular descriptors. Returns None if the molecule cannot be generated.  
      """  
  
      # Generate a molecule object from the SMILES string  
      mol = Chem.MolFromSmiles(smiles)  
      if mol is not None:  
          # If the molecule object is successfully created, calculate molecular descriptors  
          descriptors = {}  
          # Iterate over all molecular descriptor functions  
          for descriptor_name, descriptor_function in Descriptors.desclist:  
              try:  
                  # Attempt to calculate each descriptor and store its value in the dictionary  
                  descriptors[descriptor_name] = descriptor_function(mol)  
              except:  
                  # If calculation fails, set the descriptor's value to None  
                  descriptors[descriptor_name] = None  
          return descriptors  
      else:  
          # If the molecule object cannot be generated, return None  
          return None  
  
      # Apply the calculate_all_descriptors function to each SMILES string in the 'Smiles' column of Tg_df  
      descriptors_df = Tg_df['Smiles'].apply(lambda x: calculate_all_descriptors(x))  
      # Convert the list of descriptor dictionaries into a DataFrame where each row represents the descriptors for a molecule  
      descriptors_df = pd.DataFrame(descriptors_df.tolist())  
      # Display the resulting DataFrame with molecular descriptors  
      descriptors_df
```

**Define a function that
calculates molecular
descriptors using RDKit**

**Apply the function on our
dataset**

Descriptors

The specific meanings of the descriptors in the dataframe can be found at the following URL : <https://www.rdkit.org/docs/Cartridge.html#descriptors>

	MaxAbsEStateIndex	MaxEStateIndex	MinAbsEStateIndex	MinEStateIndex	qed	SPS	MolWt	HeavyAtomMolWt	ExactMolWt	NumValenceElectrons	...
0	1.750000	1.750000	0.875000	0.875000	0.355446	20.000000	14.027	12.011	14.015650	6	...
1	2.395833	2.395833	0.750000	0.750000	0.414720	25.666667	42.081	36.033	42.046950	18	...
2	2.332824	2.332824	0.743056	0.743056	0.451401	22.250000	56.108	48.044	56.062600	24	...
3	2.310000	2.310000	0.734306	0.734306	0.476641	20.200000	70.135	60.055	70.078250	30	...
4	2.374491	2.374491	0.699074	0.699074	0.465496	21.400000	70.135	60.055	70.078250	30	...
...	...	...	...	...	...	...	...	...	...	...	...
6901	14.230116	14.230116	1.115390	-5.853856	0.764710	44.411765	270.119	264.071	270.029091	100	...
6902	14.610956	14.610956	0.457778	-6.005614	0.762588	33.809524	330.190	319.102	330.066619	124	...
6903	14.547685	14.547685	0.782397	-6.107007	0.783068	34.900000	316.163	307.091	316.050969	118	...
6904	6.761067	6.761067	0.026076	-0.514062	0.096597	23.661538	855.179	792.683	854.481129	328	...
6905	6.791436	6.791436	0.030349	-0.482500	0.085673	23.313433	883.233	816.705	882.512429	340	...

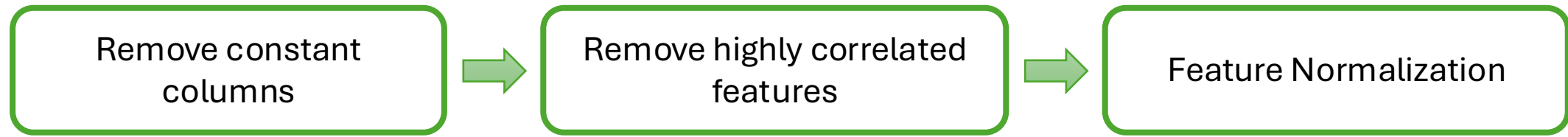
6906 rows x 217 columns

The number of descriptors generated by RDKit



alvaDesc 5471 descriptors

Descriptors



Descriptors



```
✓ [25] # Remove Constant Columns  
0s descriptors_df_noconstant = descriptors_df.loc[:, (descriptors_df != descriptors_df.iloc[0]).any()]  
descriptors_df_noconstant = descriptors_df_noconstant.dropna(axis=1, how='any')  
# report number of columns  
len(descriptors_df_noconstant.columns)
```

⇌ 194



**The number of descriptors after remove constant columns
217 → 194**

Descriptors

Remove constant
columns



Remove highly correlated
features



Feature Normalization

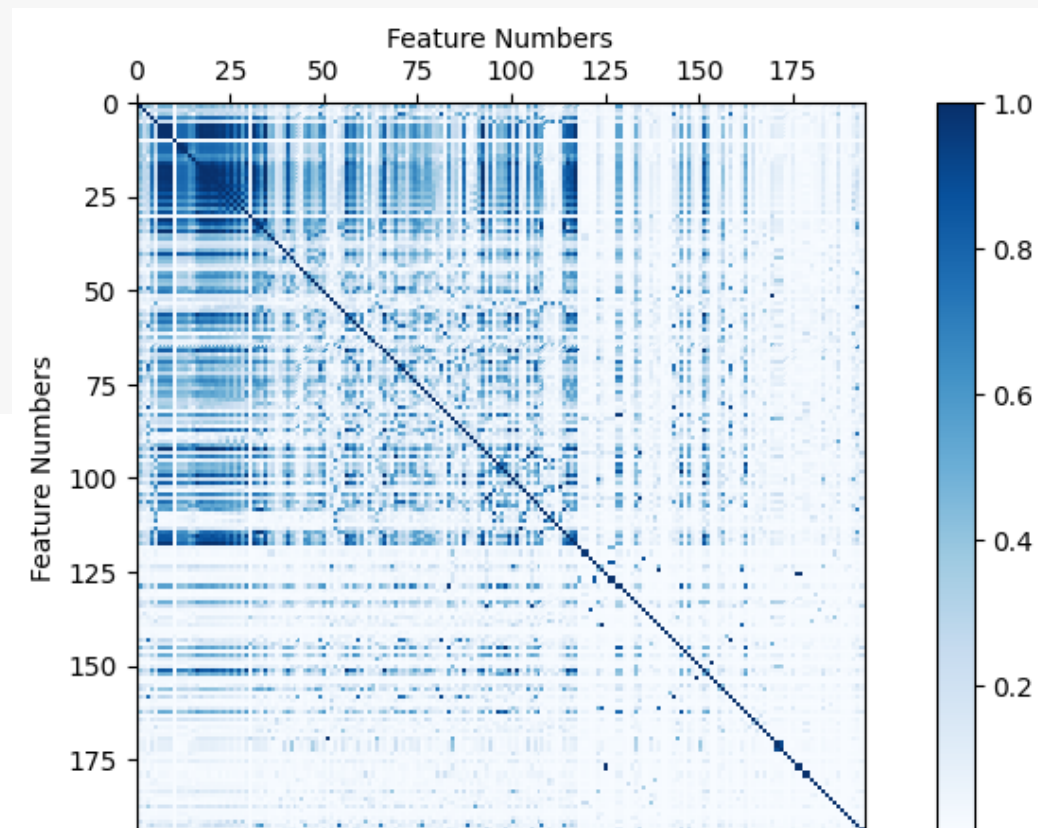
```
✓ [26] # Remove Highly correlated Features  
1s # using notes here for methodology: https://chrisalbon.com/machine\_learning/feature\_selection/drop\_highly\_correlated\_features/  
  
descriptors_corr_df = descriptors_df_noconstant.corr(method="pearson").abs()
```

→ Compute the correlation between every pair of descriptors

```
✓ # before removing correlated features  
5s fig1, ax1 = plt.subplots(figsize=(10,5))  
c = ax1.pcolor(descriptors_corr_df, cmap="Blues")  
ax1.set_ylim(ax1.get_ylim()[::-1])  
ax1.xaxis.set_ticks_position('top')  
ax1.xaxis.set_label_position('top')  
ax1.set_xlabel('Feature Numbers')  
ax1.set_ylabel('Feature Numbers')  
ax1.set_aspect('equal')  
plt.colorbar(c, ax=ax1)  
plt.show()
```



Plot the correlation between every pair of
descriptors



Descriptors

Remove constant columns

Remove highly correlated features

Feature Normalization

```
[28] # Filter the features with correlation coefficients above 0.95
upper = descriptors_corr_df.where(np.triu(np.ones(descriptors_corr_df.shape), k=1).astype(bool))
to_drop = [column for column in upper.columns if any(upper[column] > 0.95)]
descriptors_df_lowcorr = descriptors_df_noconstant.drop(columns=to_drop)
# recalculate the correlation matrix so we can compare
descriptors_corr_df_update = descriptors_df_lowcorr.corr(method="pearson").abs()
```

```
# plot correlation after removing highly correlated features
```

```
fig1, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
c1 = ax1.pcolor(descriptors_corr_df, cmap="Blues")
ax1.set_ylim(ax1.get_ylim()[::-1])
ax1.xaxis.set_ticks_position('top')
ax1.xaxis.set_label_position('top')
ax1.set_xlabel('Feature Numbers')
ax1.set_ylabel('Feature Numbers')
ax1.set_aspect('equal')

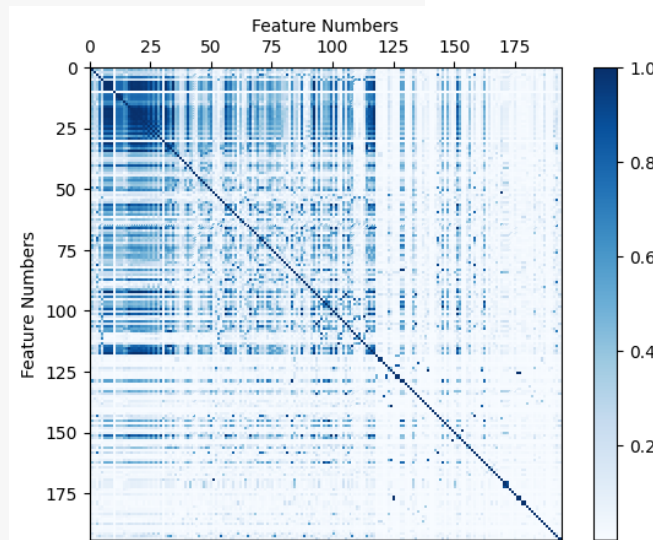
plt.colorbar(c1, ax=ax1)

c2 = ax2.pcolor(descriptors_corr_df_update, cmap="Blues")
ax2.set_ylim(ax2.get_ylim()[::-1])
ax2.xaxis.set_ticks_position('top')
ax2.xaxis.set_label_position('top')
ax2.set_xlabel('Feature Numbers')
ax2.set_ylabel('Feature Numbers')
ax2.set_aspect('equal')
plt.colorbar(c2, ax=ax2)
plt.show()
```

```
len(descriptors_df_lowcorr.columns)
```

148

Remove the descriptors with correlation coefficients above 0.95



Descriptors with high correlation (shown as dark blue regions) have been removed

The number of descriptors
217 → 194 → 148

Descriptors

Remove constant
columns



Remove highly correlated
features



Feature Normalization

```
max_values = descriptors_df_lowcorr.max()

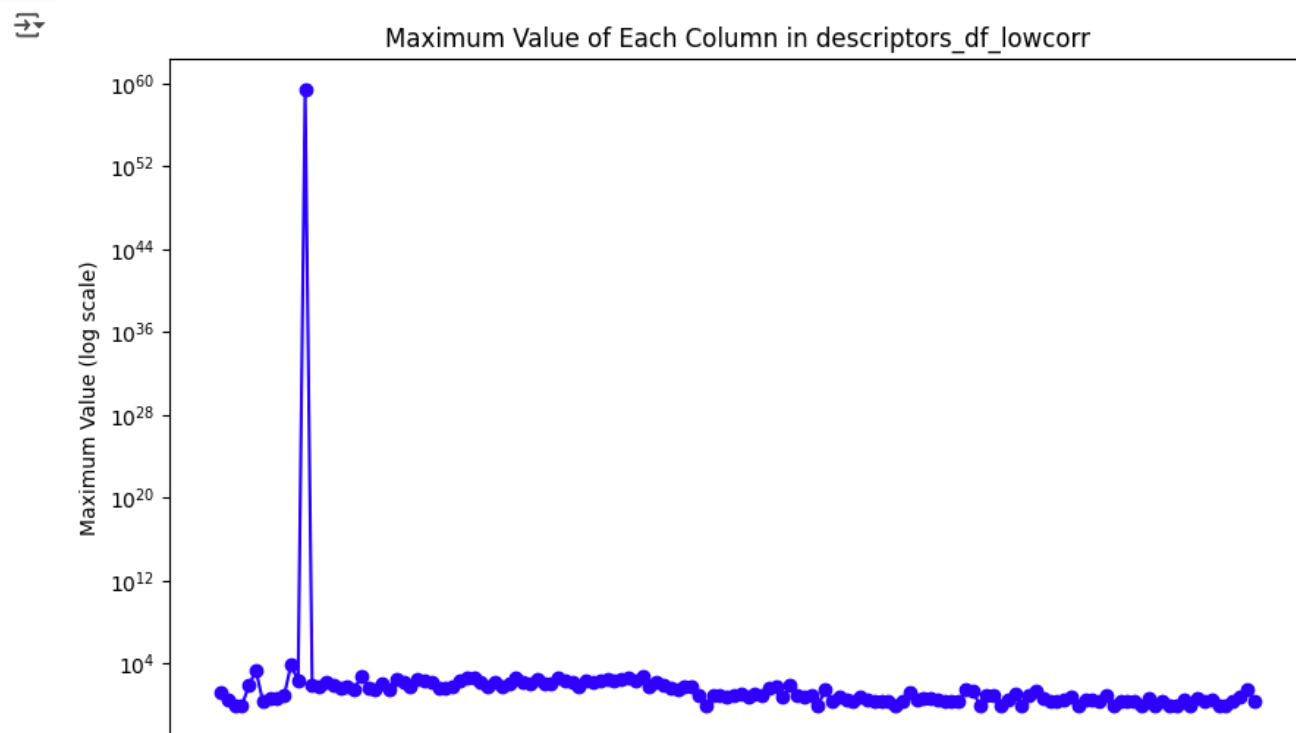
plt.figure(figsize=(10, 6))
plt.plot(max_values, marker='o', linestyle='-', color='b')
plt.yscale('log')

plt.title('Maximum Value of Each Column in descriptors_df_lowcorr')
plt.ylabel('Maximum Value (log scale)')

plt.xticks([])

plt.show()
```

**Compute and visualize the maximum value
of each descriptor**



**Descriptors vary greatly in scale, with
dramatic differences in their value ranges**

Descriptors

Remove constant
columns



Remove highly correlated
features



Feature Normalization

```
[33] # Apply Min-Max scaling to normalize the descriptors data
      # MinMaxScaler scales the features to a range between 0 and 1
      minmax_descriptors = MinMaxScaler().fit_transform(descriptors_df_lowcorr)
      # Convert the normalized descriptors back into a DataFrame
      # The column names are preserved from the original DataFrame
      minmax_descriptors_df = pd.DataFrame(minmax_descriptors, columns=descriptors_df_lowcorr.columns)
```

**Apply Min-Max scaling to
normalize the descriptors data**

```
[35] max_values = minmax_descriptors_df.max()

      plt.figure(figsize=(10, 6))
      plt.plot(max_values, marker='o', linestyle='--', color='b')

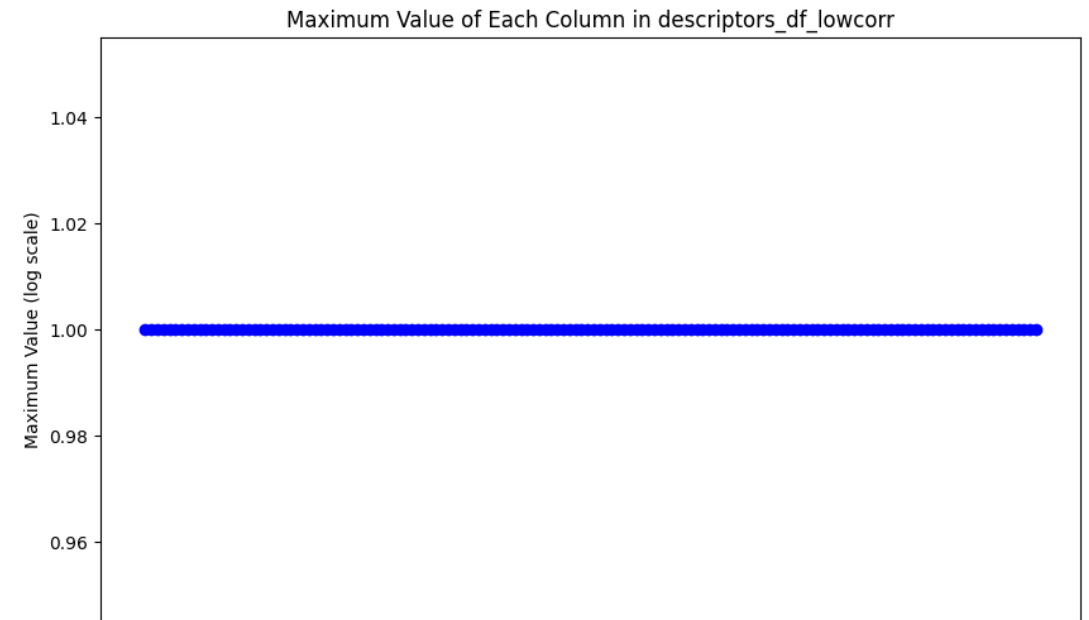
      plt.title('Maximum Value of Each Column in descriptors_df_lowcorr')
      plt.ylabel('Maximum Value (log scale)')

      plt.xticks([])

      plt.show()
```



Visualize the maximum value of each descriptor



All descriptor values have been well normalized

Question Time

Thank you!

Tianle Yue (tyue4@wisc.edu)